Policy-Driven Controls: From the Policy Document to Audit Decisions

Section draft for the AuditIQ paper

I. Policy-Driven Controls

A. Motivation

A procurement audit applies thresholds: how far an invoice may differ from its purchase order (PO), how many invoices a vendor may submit on one day, how much each role may approve. In the prototype from which this work started, these thresholds were held in three places. The policy stated them in prose; the prompt of one agent repeated one of them as a literal number; and rule checks were tools that an agent could choose to call. A change of policy therefore meant editing prompts and code, nothing ensured that the edit agreed with the document, and whether a check ran at all depended on a model. An auditor could not be shown that the system applies the policy in force.

B. Method

We make the policy document the input of the system and remove every threshold from code and prompts.

Policy as data. A policy version is a tuple $P_v = (K, s, \kappa, \theta)$: a set of clause identifiers K; a severity $s(k)$ for each clause; a kind $\kappa(k) \in \{\text{control}, \text{judgement}\}$, which states whether the clause can be decided by comparing fields or needs reading; and parameters θ_k, such as the tolerance of the amount match or the approval limit of each role. The tuple is stored as a clause registry. Our policy has sixteen clauses, twelve of kind control, with fourteen parameter values.

Controls. A control is a function $c_k(x; \theta_k)$ of a transaction x (an invoice with its PO, goods receipt, vendor record, approval log and the vendor's other invoices) and of the parameters of its clause. It returns either nothing or a finding that cites k, carries $s(k)$, and quotes the field values on which it rests. A control contains no constant of its own. Twelve controls implement the twelve control clauses and run before any language model; amounts are compared in integer paise, so rounding cannot move a decision at a boundary.

From document to registry. A policy arrives as a PDF document D. The system (i) extracts every text line with its font size, position and page, and segments the document into numbered rules and other paragraphs from this layout; (ii) for each parameter, matches one pattern against the text of the rule that states it, for example the sentence "a difference of up to 1% of the PO amount is tolerated", and reads the severity of each rule from the rule itself; (iii) builds the registry $\hat{P} = (K, \hat{s}, \kappa, \hat{\theta})$ with $\hat{\theta} = E(D)$. The set K and the kinds κ come from a reference version, which records what the system is able to execute.

Refusal. The extraction is by pattern and not by a language model, so a value is either found in the expected sentence or reported as missing. A document is refused, and nothing is stored, when a clause the system executes is absent, when a threshold cannot be found in the wording of its rule, when the document contains a rule for which the system has no check, when a rule is stated more than once, when the approval table lacks a role, or when a value is not meaningful. The reasons are reported to the user.

Adoption. An accepted document is stored as a new policy version together with its passages and their page numbers, so that every later finding cites the rule, the document and the page. Activating the version changes the parameters every control reads. No code and no prompt is edited.

C. Implementation

The feature is written in Python 3.11. PDF text and layout are read with PyMuPDF; the policy documents of the experiment are produced with ReportLab from the registry and a text template, so that the text and the parameters of a version cannot differ; parameters are extracted with regular expressions; policy versions, documents, findings and the audit trail are stored in SQLite or PostgreSQL behind one interface; the controls run as the first stage of a LangGraph pipeline. The dataset generator and its validator share no code with the controls, which a test enforces. No language model is used in this feature.

II. Experiment: Policy Change

A. Question

When an organisation changes its policy and supplies only the new document, does the system decide every invoice as the new policy requires, and how wrong is a system that is not updated?

B. Setup

Policies. Starting from the reference policy (version 2.5), we derive fourteen variants. Eleven change one threshold: the amount tolerance (1% to 0.5%, 2% and 5%), the daily invoice count per vendor (3 to 2 and 5), the Manager approval limit (100,000 to 50,000 and 150,000), the Senior Manager limit (500,000 to 250,000), the round-amount multiple (10,000 to 5,000 and 50,000), and the working week (Saturday becomes a business day). Three change several at once: a strict policy, a lenient policy, and the second version of our original policy-swap design. Each variant is rendered as a three-page PDF, which is all the system receives.

Ledgers. Five synthetic ledgers of AuditIQ-Bench with 40,000 invoices in total: four of 5,000 invoices and one of 20,000, each with purchase orders, goods receipts, vendors and approval logs.

Systems. Policy-driven is the controls under the registry extracted from the variant's PDF. *Frozen* is the same controls with the thresholds of version 2.5; it stands for a system whose thresholds are written into prompts and code, until someone edits it.

Ground truth. For each variant, the set $Y_P(x)$ of clauses that an invoice violates is recomputed from the ledger tables by the validator of the dataset generator. It is a second implementation of the written rules and shares no code with the controls.

Measures. Policy adherence is the share of invoices for which the set of cited clauses equals the ground truth of the policy in force,

$$A(P) = \frac{1}{N} \sum_{x} \mathbf{1}\big[\{k(f) : f \in F(x)\} = Y_P(x)\big]. \qquad (1)$$

For the frozen system we also count the findings that the policy in force does not support, the exceptions it does not raise, and the invoices it routes wrongly under the count-based score with threshold 50: escalated although the policy would approve them, or approved although the policy requires review.

C. Results

Reading the documents. All fourteen documents were accepted. The system read 196 of 196 parameter values correctly and recovered 224 of 224 rules word for word. Reading one document took less than a tenth of a second (0.01 to 0.09 seconds, the first including start-up).

Adherence. Table I gives the result for each policy, pooled over the five ledgers. The policy-driven controls decided every invoice as the policy in force requires: no wrong decision in 560,000 invoice audits. Under the reference version itself the controls and the independent recomputation agree on all 40,000 invoices.

Cost of a frozen system. The number of invoices whose correct decision changes ranges from none (round-amount multiple lowered to 5,000) to 12,274 of 40,000 (30.7%) under the strict policy (Fig. 1). The direction of the change decides the kind of error. A tightening leaves a frozen system silent: with the Manager limit lowered to 50,000 it does not raise 7,677 exceptions and approves 7,514 invoices that the policy in force sends to review; under the strict policy it approves 12,012. A loosening costs reviewer time instead: under version 2.6 a frozen system raises 302 findings that the policy no longer supports and escalates 302 invoices needlessly.

Held-out documents. The fourteen variants were written by hand. To test values that played no part in building or checking the reader, we drew twenty further variants at random with a fixed seed, each changing between one and five values, and applied each to the same 40,000 invoices. The system read 280 of 280 values and 320 of 320 rules correctly, adopted all twenty documents, and the policy-driven controls made no wrong decision in 800,000 invoice audits. A frozen system

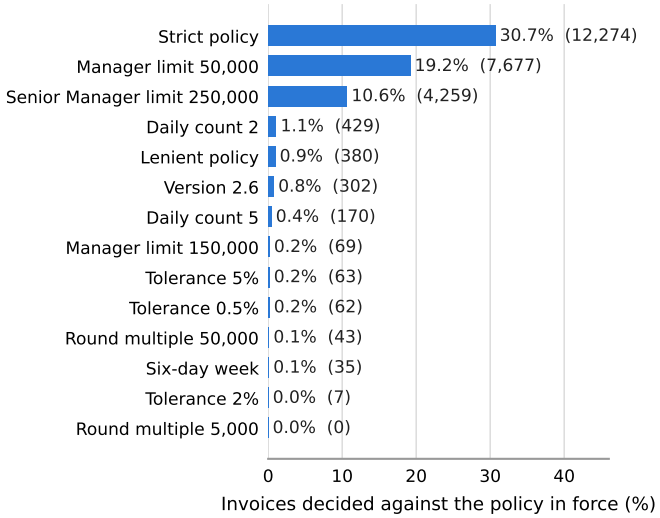

Fig. 1. Share of the 40,000 invoices that a system with thresholds frozen at version 2.5 decides against the policy in force, for each policy variant. The policy-driven controls decide none against it.

decided up to 7,953 of the invoices (19.9%) against the policy in force.

Robustness of the reader. We edited the reference policy in sixteen ways and read each edited PDF. All seven edits that keep every value (extra prose, rules in another order, long rules, page breaks, a reordered approval table, a changed severity, Indian digit grouping) were read correctly, and all seven edits that make the document impossible to execute were refused with the reason. Two edits make it ambiguous: a rule stated twice with two tolerances, and an approval table without one role. The first run adopted both; we added both checks to the reader, with regression tests, and on the second run all sixteen documents were handled as required.

In the application. We repeated one change through the deployed path. On the ledger of 20,000 invoices, audited under version 2.5, 240 invoices awaited review and 300 findings were stored. After the strict policy document was adopted with one command and the ledger audited again, 6,353 invoices awaited review and 6,546 findings were stored, 6,060 of them under the approval-limit clause. Reading and storing the document took 0.03 seconds, and the audit of 20,000 invoices, with every result, finding and trail entry stored, took 43 seconds on a laptop (50 seconds before the change). No file of the system was edited between the two runs.

Throughput. In memory, the controls audited 560,000 invoices in 14 seconds in a single process (about 41,000 invoices per second on an Intel i5-1235U), without any model call.

D. Discussion and Limitations

The adherence of 1.000 must be read for what it is. The controls and the ground truth are two implementations of the same written rules by the same authors; that they agree on every invoice under every policy shows that each document was read correctly and that no control keeps a threshold of

TABLE I
POLICY CHANGE: ADHERENCE TO THE POLICY IN FORCE ON 40,000 INVOICES PER POLICY

Policy variant	Decisions moved	Policy-driven	Frozen at v2.5			
		Adherence	Adherence	Unsupported findings	Missed exceptions	Missed escalations
Tolerance 0.5%	62	1.000	0.998	0	62	62
Tolerance 2%	7	1.000	1.000	7	0	0
Tolerance 5%	63	1.000	0.998	63	0	0
Daily count 2	429	1.000	0.989	0	429	428
Daily count 5	170	1.000	0.996	170	0	0
Manager limit 50,000	7,677	1.000	0.808	0	7,677	7,514
Manager limit 150,000	69	1.000	0.998	69	0	0
Senior Manager limit 250,000	4,259	1.000	0.894	0	4,259	4,161
Round multiple 5,000	0	1.000	1.000	0	0	0
Round multiple 50,000	43	1.000	0.999	43	0	0
Six-day week	35	1.000	0.999	35	0	0
Strict policy	12,274	1.000	0.693	0	12,427	12,012
Lenient policy	380	1.000	0.991	380	0	0
Version 2.6	302	1.000	0.992	302	0	0

its own. It is a verification of the mechanism, not a detection result, and it says nothing about whether the rules are the right ones.

The quantities that are not fixed by construction are those of the frozen system. They show that ordinary revisions of a procurement policy move between a handful and a third of the decisions on these ledgers, and that a tightening turns into missed escalations. A system that must be edited by hand to follow its policy carries this risk for as long as the edit is late or wrong.

Four limits apply. First, pattern extraction reads only the wording of one template and refuses other documents with reasons. A model-assisted path for policies in their own wording was measured separately on six short documents: a local 7B model proposed 80 of 84 values correctly, a check by code verified 56 of them and held back 24 right ones for the reviewer, and two wrong values passed the check, so the confirming person remains a required safeguard. Second, the system executes only rules for which it has a control; a new kind of rule needs a new control. Third, the ledgers are synthetic, so the counts describe how these ledgers react to these changes and not the books of a particular organisation. Fourth, the experiment covers the twelve rule-defined clauses; the four judgement clauses are assigned to language-model agents, which have been run only on a pilot of 20 invoices and a sample of 34 invoices and whose accuracy on the full benchmark we have not yet measured.